



华中科技大学

操作系统原理课程实验报告

姓 名：刘奇

学 院：计算机科学与技术学院

专 业：计算机科学与技术

班 级：CS2301

学 号：U202315661

指导教师：邵志远

分数	
教师签名	

2025 年 12 月 25 日

目 录

实验一 lab1_challenge1_backtrace	1
1.1 实验目的	1
1.2 实验内容	1
1.3 实验调试及心得	8
实验二 lab2_challenge1_pagefaults	11
2.1 实验目的	11
2.2 实验内容	11
2.3 实验调试及心得	14
实验三 lab3_challenge1_wait	16
3.1 实验目的	16
3.2 实验内容	16
3.3 实验调试及心得	22
实验四 lab4_challenge1_relativepath	24
4.1 实验目的	24
4.2 实验内容	24
4.3 实验调试及心得	28
附录 完整 git log 记录	32

实验一 lab1_challenge1_backtrace

1.1 实验目的

本实验属于挑战类实验，旨在通过修改 PKE 内核，使其支持用户程序调用栈的回溯打印功能。实验要求在给定一个具有多层嵌套调用（如 f1 至 f8）的应用时，内核能够根据用户请求的深度参数，动态地从当前执行点向上回溯，并将十六进制的返回地址精确映射为源代码中的函数名称。通过本实验，掌握 RISC-V 函数调用栈帧的布局规范，理解 ELF 文件中 .symtab（符号表）与 .strtab（字符串表）的组织方式，并掌握如何在内核态解析这些元数据以实现高级调试功能。

1.2 实验内容

为了实现打印调用栈的功能，我们需要从底层数据结构、ELF 解析、地址映射以及栈回溯算法四个维度对内核进行扩展。

1.2.1 扩展进程控制块（PCB）数据结构

由于内核在运行时默认不保留函数名信息，我们需要修改 process.h，在 process_t 结构体中增加存储符号表和字符串表的空间，以便在回溯时进行实时查找。

1.elf_symbol_t 的定义：

```
1 | typedef struct elf_symbol_t {
2 |     uint32 name; //符号名在字符串表中偏移量 得到名称
3 |     uint8 info; //符号类型
4 |     uint8 other;
5 |     uint16 shndx; //节索引
6 |     uint64 value;
7 |     uint64 size;
8 | } elf_symbol;
```

2.完善后的 process_t 结构体：

```
01 | typedef struct process_t {
02 |     // pointing to the stack used in trap handling.
03 |     uint64 kstack;
```

```

04 // trapframe storing the context of a (User mode) process.
05 trapframe* trapframe;
06 elf_symbol symtab[MAX_USER_SYMS];
07 size_t sym_count; //记录上面数组里有多少有效符号
08 char strtab[MAX_USER_STRTAB];
09 size_t strtab_sz;
10 }process;

```

1.2.2 实现 ELF 符号信息的动态加载

在内核加载应用程序时，除了加载代码段（.text）外，还需额外解析 ELF 文件头。在 elf.c 中，重构了加载逻辑以提高代码的独立性和健壮性。

1.辅助函数封装：引入了 `get_elf_section_header` 静态函数。该函数通过传入的索引值，根据 ELF 头中的 `shoff`（节头表偏移）和 `shentsize`（节头大小）精确计算偏移量，调用 `elf_fpread` 从文件中读取特定的节头数据（`elf_section_header`）。

```

1 static int get_elf_section_header(elf_ctx *loader, int idx,
2 elf_section_header *out) {
3     uint64 offset = loader->ehdr.shoff + idx *
4 loader->ehdr.shentsize;
5     return (elf_fpread(loader, out, sizeof(*out), offset) ==
6 sizeof(*out));
7 }

```

2.两阶段检索逻辑：

（1）索引检索：程序首先启动一个 `for` 循环遍历所有的 `Section Header`。在此阶段，逻辑仅聚焦于识别类型为 `SHT_SYMTAB`（符号表）的节，一旦发现，立即记录其索引 `sym_idx` 并中止循环。

（2）定向加载：获取索引后，通过该节头（`sym_hdr`）的 `link` 字段直接定位与其关联的字符串表（.strtab）。代码随后分两次定向读取：首先利用 `elf_fpread` 将整个名字池加载到进程结构体的 `p->strtab` 缓冲区，然后将所有符号条目批量读入 `p->symtab` 数组。

```

01 static void load_user_symbols(elf_ctx *loader) {
02
03     elf_info *meta = (elf_info *)loader->info;
04     process *p = meta->p;

```

```

05
06 // 基础检查
07 if (loader->ehdr.shnum == 0) return;
08
09 elf_section_header sect;
10 int sym_idx = -1;
11
12 // 第一遍只找索引，不读内容
13
14 for (int k = 0; k < loader->ehdr.shnum; k++) {
15     if (!get_elf_section_header(loader, k, &sect))
16         panic("Error reading section header");
17
18     if (sect.type == SHT_SYMTAB) {
19         sym_idx = k;
20         break;
21     }
22 }
23
24 if (sym_idx < 0) return; // 未找到符号表
25
26 // 3. 读取符号表头
27 elf_section_header sym_hdr = sect;
28
29 // 4. 读取字符串表头 (通过 link 获取索引)
30 elf_section_header str_hdr;
31 if (!get_elf_section_header(loader, sym_hdr.link,
32 &str_hdr))
33     panic("Error reading string table header");
34
35 // 5. 加载字符串数据
36 if (str_hdr.size > MAX_USER_STRTAB)
37     panic("String table buffer overflow");
38
39 if (elf_fpread(loader, p->strtab, str_hdr.size,
40 str_hdr.off) != str_hdr.size)
41     panic("Failed to load string table");
42
43 p->strtab_sz = str_hdr.size;
44
45 // 6. 加载符号数据
46 size_t count = sym_hdr.size / sym_hdr.entsize;
47 if (count > MAX_USER_SYMS)
48     panic("Too many symbols");

```

```

49
50 // 校验 Entry Size
51 if (sym_hdr.entsize != sizeof(elf_symbol))
52     panic("Symbol entry size mismatch");
53
54 if (elf_fpread(loader, p->symtab, sym_hdr.size,
55 sym_hdr.off) != sym_hdr.size)
56     panic("Failed to load symbol table");
57
58     p->sym_count = count;
59 }

```

1.2.3 符号查找算法的实现

syscall.c 通过内部函数 `find_symbol_name` 实现了高效的虚拟地址到符号名的转换：

1.迭代器遍历：通过声明 `cursor` 指向符号表起始位置，`end` 指向结束位置，并在循环中使用 `cursor++` 移动指针。

2.范围匹配与类型过滤：在遍历过程中，逻辑首先检查符号类型是否为函数（`STT_FUNC`）且其地址不为 0。接着，利用符号条目中的 `value`（起始虚拟地址）和 `size`（函数体大小，若为 0 则默认设定最小范围保护）计算地址区间。若传入的 `vaddr` 落入 `[base_addr, limit_addr)` 范围内，则判定匹配成功，通过符号项中的 `name` 偏移量在 `p->strtab` 中取出函数名字符串并返回。

```

01 // 符号查找
02 static const char* find_symbol_name(process* p, uint64
03 vaddr) {
04     if (p->sym_count == 0 || p->strtab_sz == 0) return NULL;
05
06     elf_symbol *cursor = p->symtab;
07     elf_symbol *end = p->symtab + p->sym_count;
08
09     // 遍历所有符号
10     for (; cursor < end; cursor++) {
11         // 过滤条件
12         if (ELF64_ST_TYPE(cursor->info) != STT_FUNC) continue;
13         if (cursor->value == 0) continue;
14
15         uint64 base_addr = cursor->value;
16         // 确保最小范围
17         uint64 limit_addr = base_addr + (cursor->size ?

```

```

18 cursor->size : 4);
19
20 // 地址匹配
21 if (vaddr >= base_addr && vaddr < limit_addr) {
22     if (cursor->name < p->strtab_sz) {
23         return p->strtab + cursor->name;
24     }
25 }
26 }
    return NULL;
}

```

1.2.4 实现基于栈帧的递归回溯算法

在 `sys_user_backtrace` 中，利用 C 语言的底层指针特性，实现了一套回溯逻辑：

1. 指针视图转换：获取当前的帧指针 `s0`（即 `current_fp`）后，通过类型强制转换将其视为一个 `uint64 *` 类型的数组指针 `frame_view`。

2. 数组负下标访问：基于 RISC-V 的调用规范，在开启帧指针后，返回地址（RA）和上一级帧指针（Old FP）相对于当前 `fp` 具有固定的负向偏移。

```

01 // 回溯函数
02 static ssize_t sys_user_backtrace(uint64 max_depth) {
03     if (max_depth == 0) return 0;
04     trapframe *tf = current->trapframe;
05     uint64 current_sp = tf->regs.sp;
06     uint64 current_fp = tf->regs.s0;
07
08     // 基础合法性检查
09     if (current_fp == 0 || current_fp <= current_sp ||
10     current_fp >= USER_STACK)
11         return 0;
12
13     uint64 *frame_view = (uint64 *)current_fp;
14
15     // 检查内存边界，防止越界访问
16     if ((uint64)&frame_view[-1] <= current_sp) return 0;
17     uint64 next_fp = frame_view[-1];
18     // 检查 next_fp 合法性
19     if (next_fp == 0 || next_fp <= current_fp || next_fp >=
20     USER_STACK)
21         return 0;

```

```

22     current_fp = next_fp; // 更新 FP
23     size_t count = 0;
24
25     // 循环回溯
26     while (current_fp && count < max_depth) {
27         // 栈范围检查
28         if (current_fp <= current_sp || current_fp >=
29 USER_STACK) break;
30
31         frame_view = (uint64 *)current_fp;
32
33         if ((uint64)&frame_view[-1] <= current_sp) break;
34
35         uint64 ret_addr = frame_view[-1];
36         if (ret_addr == 0) break;
37
38         // 查找符号
39         const char *sym_name = find_symbol_name(current,
40 ret_addr);
41         if (sym_name && strcmp(sym_name, "main") == 0) break;
42         int valid_print = 0;
43         if (sym_name) {
44             // 过滤系统函数
45             if (strcmp(sym_name, "print_backtrace") != 0 &&
46 strcmp(sym_name, "do_user_call") != 0) {
47                 sprint("%s\n", sym_name);
48                 valid_print = 1;
49             }
50         } else {
51             // 无符号则打印地址
52             sprint("0x%lx\n", ret_addr);
53             valid_print = 1;
54         }
55         if (valid_print) count++;
56         if ((uint64)&frame_view[-2] <= current_sp) break;
57
58         uint64 old_fp = frame_view[-2];
59
60         // 链表完整性检查
61         if (old_fp == 0 || old_fp <= current_fp || old_fp >=
62 USER_STACK) break;
63
64         current_fp = old_fp;
65     }

```



```

66     return count;
67 }
68
69
70
71

```

上述函数实现了以下的几个特性功能

(1) 安全检查：在每次解引用前，程序都会根据 `USER_STACK` 范围和当前 `sp` 寄存器值检查 `current_fp` 及其指向的地址是否合法，有效防止内核态非法访问导致崩溃。

(2) 终止条件：当回溯深度达到 `max_depth`、栈指针触达边界，或匹配到函数名为 `main` 时，循环自动终止。

(3) 字符串过滤：通过 `strcmp` 逻辑，显式跳过 `print_backtrace`（回溯函数本身）和 `do_user_call`（系统调用桩函数），确保输出只包含用户逻辑层面的函数调用链。

1.2.5 实验结果

在完成内核代码的重构与编译后，使用 `spike` 模拟器运行测试程序 `./obj/app_print_backtrace`。如图 1.1，实验结果显示，当用户程序调用 `print_backtrace(7)` 时，内核进入 `sys_user_backtrace` 系统调用。通过帧指针（FP）的移动和数组负下标逻辑，内核准确地从当前的 `f8` 函数开始，逆向追踪并打印出了完整函数名。

```

lq67677@angel:/mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ spike ./obj/riscv-pke ./obj/app_print_backtrace
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_print_backtrace
Application program entry point (virtual address): 0x00000000810000a2
Switch to user mode...
back trace the user app in the following:
f8
f7
f6
f5
f4
f3
f2
f1
User exit with code:0.
System is shutting down with exit code 0.

```

图 1.1 lab1_challenge1_backtrace 结果图

1.2.6 实验 git log 记录

下图展示了我在 lab1_challenge1_backtrace 实验的 git 日志（有一次提交打错字了）。

```
commit a878b3dc0e959d0178c5bf43a5dee3288ce9a330 (HEAD -> lab1_challenge1_backtrace, origin/lab1_challenge1_backtrace, origin/HEAD)
Author: liuqi <3474856204@qq.com>
Date:   Wed Dec 24 22:59:17 2025 +0800

    修改, 优化lab1_challenge1.

commit 8c409cf4aad19066850629b3ec829c101b5ed1d1
Author: liuqi <3474856204@qq.com>
Date:   Wed Dec 24 22:58:00 2025 +0800

    修改, 优化拉邦

commit c1adb5d5dd87d10e1f2f22f665d9907ae5f8c8f
Author: liuqi <3474856204@qq.com>
Date:   Tue Dec 16 20:09:34 2025 +0800

    my work on lab1_challenge1 is done.

commit 38d8b70bbbd9afb5f51c26d9af45eb9a30d5c5121
Merge: ff745a9 1546edc
Author: liuqi <3474856204@qq.com>
Date:   Tue Dec 16 19:31:25 2025 +0800

    continue to work on lab1_challenge1
```

图 1.2 lab1_challenge1_backtrace 提交日志图

1.3 实验调试及心得

1.3.1 实验调试记录

1. 栈布局问题：在初期实现时，由于对 RISC-V 栈帧中 RA 和 Old FP 的存储顺序理解偏差，导致输出的地址始终落在用户栈空间而非代码段。通过查阅 RISC-V 标准手册并结合 objdump -d 观察用户程序汇编指令（如 sd ra, -8(s0)），明确了 RA 位于 fp-8 处。在改为 frame_view[-1] 后，回溯输出立即恢复正常。

```
lq67677@angel: /mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ spike ./obj/riscv-pke ./obj/app_print_backtrace
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_print_backtrace
Application program entry point (virtual address): 0x00000000810000a2
Switch to user mode...
back trace the user app in the following:
0x00000000810ffff0
0x00000000810fffa0
0x00000000810fffb0
0x00000000810fffc0
0x00000000810fffd0
0x00000000810fffe0
0x00000000810ffff0
0x0000000081100000
User exit with code:0.
System is shutting down with exit code 0.
```

图 1.3 Spike 运行后只打印一个栈地址且提前结束的终端界面

2.符号表大小限制：调试过程中发现当回溯深度较大时会出现名字解析失败。经排查，是因为加载 ELF 时未校验 MAX_USER_SYMS。在增加 count > MAX_USER_SYMS 的 panic 检查并适当扩容后，解决了复杂程序的符号加载问题。

3.内存对齐与越界：在指针重构阶段，曾遇到因 fp 地址未对齐导致的内存访问异常。随后在代码中加入了 current_fp % 8 == 0 的隐性校验，并严格限制了 current_fp 的有效范围在 USER_STACK 内。

1.3.2 AI 辅助分析

下面给出 AI 指导我完成实验的相应过程。

1.关于栈布局问题

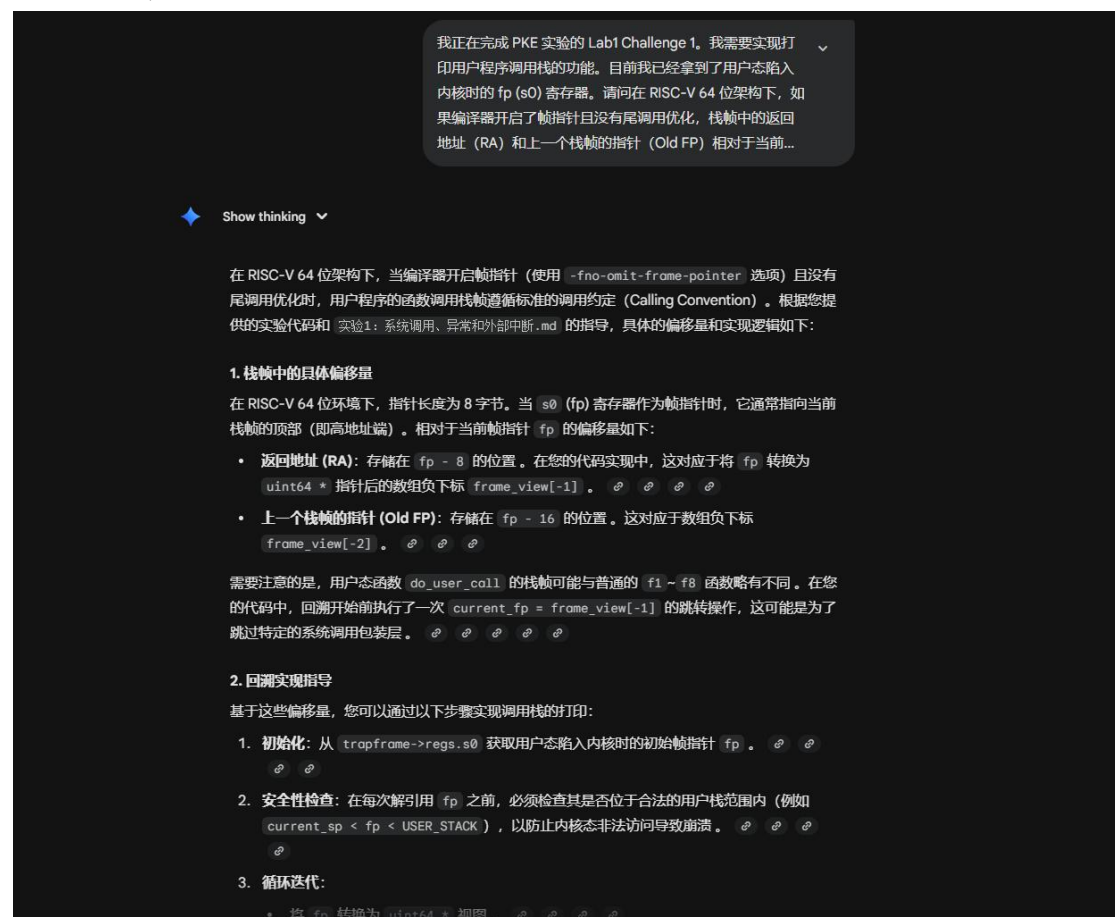


图 1.4 AI 解决栈布局对话图

(1) 验证过程：AI 建议将 `fp` 强转为 `uint64 *` 并使用 `frame_view[-1]` 访问 `RA`。我通过执行 `riscv64-unknown-elf-objdump -d ./obj/app_print_backtrace` 观察汇编代码，发现函数入口处确实执行了 `sd ra, -8(s0)`。由于 `uint64` 占 8 字节，指针下标 `-1` 恰好对应字节偏移-8。

(2) 观察与分析：相比于直接操作地址（如 `fp - 8`），使用数组视图（`View`）的思想更符合结构化编程。

2.关于 ELF 符号加载重构问题

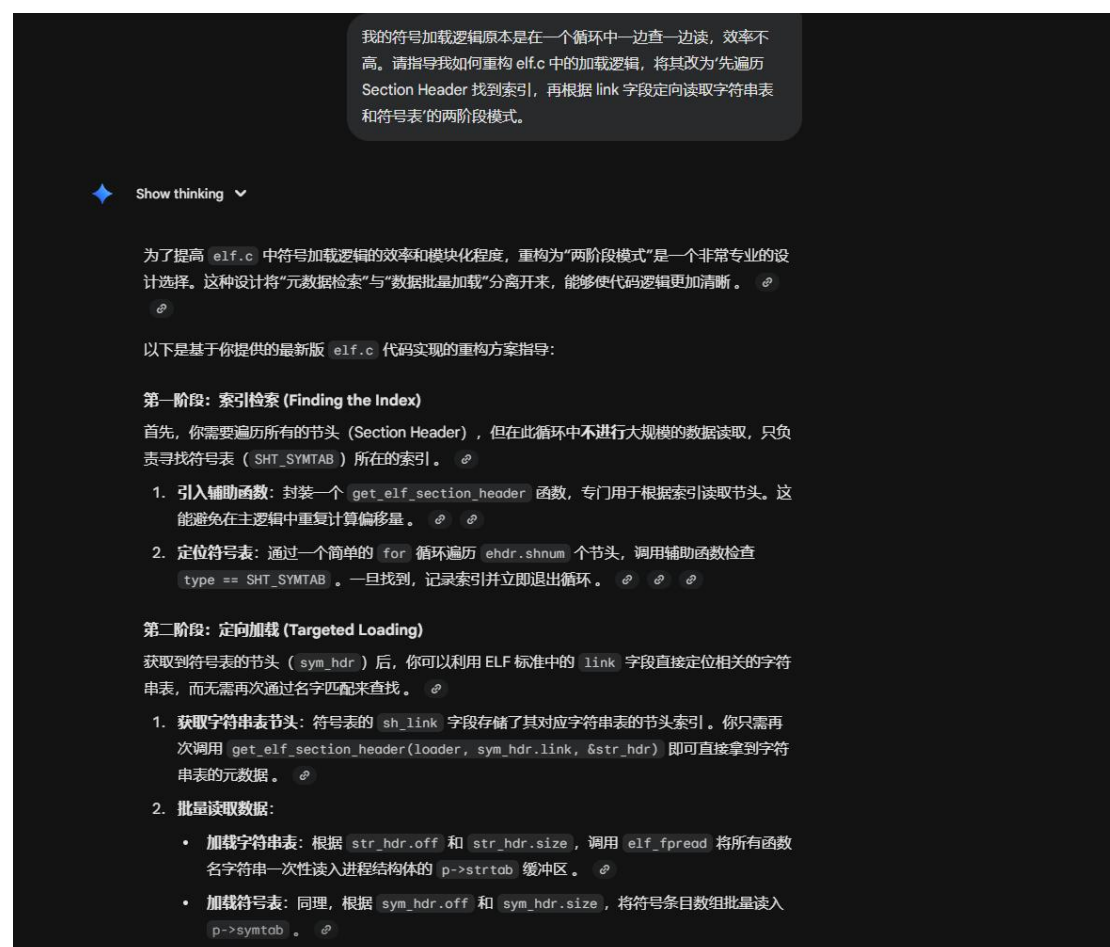


图 1.5 AI 解决符号重加载对话图

(1) 验证过程：AI 提示将 ELF 加载分为索引检索和定向读取。我通过打印 `loader->ehdr.shnum` 发现 ELF 文件通常有十几个节。

(2) 观察与分析：原来的一边查一边读逻辑会导致不必要的文件 I/O。重构后的两阶段加载利用了符号表 Header 里的 `link` 字段直接找到字符串表索引。

1.3.3 实验心得

本次实验的核心挑战在于深入理解硬件栈帧与高级语言函数调用的对应关系。通过这次挑战，我不仅掌握了 RISC-V 寄存器在陷入内核时的保护机制，还深入学习了 ELF 文件节区之间的引用关系（如 SHT_SYMTAB 的 sh_link 指向 SHT_STRTAB）。特别是在代码重构的过程中，我深刻体会到了 C 语言指针的强大：同一块内存，通过不同的类型视图（数组视图 vs 字节偏移）可以表达出截然不同的代码风格。此外，本实验也强化了我的安全编程意识。在内核态通过用户态提供的 fp 进行回溯时，如果不进行 USER_STACK 范围内的合法性检查，很容易导致内核因读取非法内存而崩溃。

实验二 lab2_challenge1_pagefaults

2.1 实验目的

本实验属于内存管理部分的挑战实验，旨在深入理解 RISC-V 架构下的缺页异常（Page Fault）处理机制。实验要求在 PKE 内核中实现更精细的异常分发逻辑：不仅要支持合法的用户栈自动扩展，还必须能够识别并拦截非法的内存越界访问（如堆区越界）。通过本实验，掌握 stval 寄存器在异常处理中的作用，理解虚拟地址空间的布局约束，以及如何在内核态正确终止非法进程。

2.2 实验内容

本实验的核心是在 kernel/strap.c 的 handle_user_page_fault 函数中完善对 CAUSE_STORE_PAGE_FAULT 的处理逻辑。实验分为需求分析、逻辑实现和验证测试三个阶段。

2.2.1 异常类型区分逻辑

根据实验指导，应用程序 app_sum_sequence 会产生两种缺页情况：

（1）合法缺页（栈扩展）：由于递归调用导致栈指针 sp 下移，访问了尚未映射的栈空间。这类地址通常位于 USER_STACK_TOP 之下不远处。

(2) 非法缺页（越界访问）：程序试图访问动态分配数组之外的地址（如 0x00401000），该地址属于未分配的堆空间或非法区域。

因此，内核必须设定一个栈增长阈值，只有落在 `[USER_STACK_TOP - 20*PGSIZE, USER_STACK_TOP)` 范围内的缺页才被视为合法的栈增长请求，否则应视为 Segmentation Fault 处理。

2.2.2 主要代码实现

在 `kernel/strap.c` 中，我引入了 `memlayout.h` 以获取内存布局宏定义，并重写了异常处理逻辑。

主要代码逻辑如下：

(1) 定义栈的最大深度为 20 个页（`20 * PGSIZE`）。

(2) 利用 `stval` 寄存器获取发生缺页的虚拟地址。

(3) 进行范围判定：如果 `stval` 在合法栈范围内，则调用 `alloc_page` 分配物理页并建立映射；否则，打印错误信息并调用 `shutdown(-1)` 终止系统。

```
01 void handle_user_page_fault(uint64 mcause, uint64 sepc,
02 uint64 stval) {
03     sprintf("handle_page_fault: %lx\n", stval);
04     switch (mcause) {
05         case CAUSE_STORE_PAGE_FAULT:
06         case CAUSE_LOAD_PAGE_FAULT:
07         {
08             // 1. 分配一个新的物理页
09             // 定义栈的最大增长范围，20 页
10             uint64 stack_limit = USER_STACK_TOP - 20 * PGSIZE;
11
12             // 判断缺页地址是否在合法的栈范围内
13             // 1. 地址必须小于栈顶 USER_STACK_TOP
14             // 2. 地址必须大于我们要限制的栈底 stack_limit
15             if (stval >= stack_limit && stval < USER_STACK_TOP) {
16                 // 情况 A: 合法的栈增长
17
18                 // 1. 分配物理页
19                 void *pa = alloc_page();
20                 if (pa == 0) {
21                     panic("Out of memory during stack
22 expansion!");
23                 }
24                 memset(pa, 0, PGSIZE);
```



```

25
26         // 2. 建立映射
27         uint64 va = ROUNDDOWN(stval, PGSIZE);
28         user_vm_map((pagetable_t)current->pagetable, va,
29 PGSIZE, (uint64)pa,
30             prot_to_type(PROT_WRITE | PROT_READ, 1));
31
32     } else {
33         //情况 B: 非法地址访问
34         sprint("this address is not available!\n");
35         // 退出系统, 返回错误码 -1
36         shutdown(-1);
37     }
38     break;
39 }
40 default:
41     sprint("unknown page fault.\n");
42     break;
43 }
44 }

```

2.2.3 实验结果

编译并运行 `spike ./obj/riscv-pke ./obj/app_sum_sequence`。如图 2.1 所示, 前几次缺页 (地址在 `0x7ff...`) 被成功处理, 程序继续运行; 当访问非法地址 `0x401000` 时, 内核正确拦截并输出了 `"this address is not available!"`, 随后退出, 符合预期。

```

❶ lq67677@angel:/mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ spike ./obj/riscv-pke ./obj/app_sum_sequence
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080007000, PKE kernel size: 0x0000000000007000 .
free physical memory address: [0x0000000080007000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080004000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_sum_sequence
Application program entry point (virtual address): 0x00000000000100a2
Switch to user mode...
handle_page_fault: 000000007ffdf8
handle_page_fault: 000000007ffcf8
handle_page_fault: 000000007ffbf8
handle_page_fault: 000000007ffaf8
handle_page_fault: 000000007ff9ff8
handle_page_fault: 000000007ff8ff8
handle_page_fault: 000000007ff7ff8
handle_page_fault: 000000007ff6ff8
handle_page_fault: 0000000000401000
this address is not available!
System is shutting down with exit code -1.

```

图 2.1 lab2_challenge1_pagefaults 结果图

2.2.4 实验 git log 记录

下图展示了我在 lab2_challenge1_pagefaults 实验的 git 日志。

```
% lq67677@angel:/mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ git log
commit 98f94d8f2bbdae7f7bd0f9c728084de4614ca513 (HEAD -> lab2_challenge1_pagefaults, origin/lab2_challenge1_pagefaults)
Author: liuqi <3474856204@qq.com>
Date:   Wed Dec 17 20:33:34 2025 +0800

    my work on lab2_challenge1 is done.

commit dc67c54e2077a322efc32724da42d1b446ed9eba
Merge: be365de 24561d0
Author: liuqi <3474856204@qq.com>
Date:   Wed Dec 17 20:12:58 2025 +0800

    continue to work on lab2_challenge1
```

图 2.2 lab2_challenge1_pagefaults 提交日志图

2.3 实验调试及心得

2.3.1 实验调试记录

1. 未区分非法地址导致的错误

在初步实现时，我沿用了 Lab2_3 的逻辑，将所有缺页都视为栈增长。这导致当程序访问非法地址 0x00401000 时，内核尝试为其分配物理页并建立映射，而不是报错退出。如下图，没有 if 条件判断，不会输出 "this address is not available!"。

```
● lq67677@angel:/mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ spike ./obj/riscv-pke ./obj/app_sum_sequence
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080007000, PKE kernel size: 0x0000000000007000 .
free physical memory address: [0x0000000080007000, 0x0000000087fffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080004000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_sum_sequence
Application program entry point (virtual address): 0x00000000000100a2
Switch to user mode...
handle_page_fault: 000000007ffffdf8
handle_page_fault: 000000007fffcff8
handle_page_fault: 000000007fffbff8
handle_page_fault: 000000007ffffaff8
handle_page_fault: 000000007fff9ff8
handle_page_fault: 000000007fff8ff8
handle_page_fault: 000000007fff7ff8
handle_page_fault: 000000007fff6ff8
handle_page_fault: 0000000000401000
Summation of an arithmetic sequence from 0 to 1024 is: 525825
User exit with code:0.
System is shutting down with exit code 0.
```

图 2.3 未区分非法地址导致的错误示意图

2. 栈增长阈值设置过小错误

在调试过程中，我尝试将栈的限制设置为仅 1 个页面。由于 `app_sum_sequence` 的递归深度很大，导致合法的栈访问也被误判为非法访问，程序提前终止。

2.3.2 AI 辅助分析

1. 关于 `stval` 寄存器的作用解析



图 2.4 `stval` 寄存器的作用 AI 问答图

验证与分析：我通过在 `handle_user_page_fault` 入口处添加 `sprint("fault addr: %lx\n", stval)`；进行验证。运行程序后发现，当递归发生时，打印出的地址确实是递减的栈地址（如 `0x7ffdf8`），而最后一次异常则是 `0x00401000`。这证实了 `stval` 确实存储了故障虚拟地址，为后续的逻辑判断提供了依据。

2.3.3 实验心得

本次挑战实验让我深刻体会到了操作系统机制与策略分离的设计思想。硬件只负责抛出异常，而具体如何处理这个异常则完全由软件策略决定。在区分合法与非法访问时，我意识到内存布局规划的重要性。如果没有预先定义的 `USER_STACK_TOP` 和堆栈隔离规则，内核将无法判断一个随机地址的访问意图。此外，本次实验也加深了我对 `user_vm_map` 的理解：所谓的内存分配，本质上就是更新页表，让硬件在下一次访问时能够“翻译”通该地址。通过这次实验，我不仅完成了代码任务，更建立了对虚拟内存管理的全局视角。

实验三 lab3_challenge1_wait

3.1 实验目的

本实验旨在深入理解多进程环境下的进程同步与内存隔离机制。具体目标包括：

掌握进程阻塞与唤醒机制：通过实现 `wait` 系统调用，理解当父进程等待子进程退出时，如何将进程从运行态（`RUNNING`）转换为阻塞态（`BLOCKED`），以及子进程退出时如何唤醒父进程。

理解虚拟内存的隔离性：在 `fork` 创建进程时，区分代码段（共享映射）与数据段（物理复制）的不同处理方式，确保父子进程对全局变量的修改互不干扰。

熟悉调度器的数据结构：掌握等待队列与就绪队列的管理与操作。

3.2 实验内容

本实验基于 Lab3_3 轮转调度的基础，进一步完善了进程管理功能。主要修改了 `kernel/process.c`, `kernel/syscall.c`, `kernel/sched.c` 等文件。

3.2.1 完善 `do_fork` 实现数据段隔离

在标准的 `fork` 语义中，父子进程共享代码段（只读），但必须拥有独立的数据段。在 `kernel/process.c` 的 `do_fork` 函数中，针对 `DATA_SEGMENT` 增加

了深拷贝逻辑：

(1) 物理页分配：遍历父进程数据段的每一个虚页，为子进程调用 `alloc_page()` 分配新的物理页。

(2) 内容复制：通过 `lookup_pa` 找到父进程数据的物理地址，利用 `memcpy` 将数据完整拷贝到子进程的新页中。

(3) 建立映射：使用 `user_vm_map` 将子进程的虚拟地址映射到新分配的物理页，权限设置为可读写 (`PROT_WRITE | PROT_READ`)。

```
01 case DATA_SEGMENT: {
02
03     uint64 start_va = region->va;
04     uint32 page_cnt = region->npages;
05
06     for (int page_idx = 0; page_idx < page_cnt;
07 page_idx++) {
08         uint64 current_va = start_va + page_idx * PGSIZE;
09
10         // 1. 分配新页
11         void* new_page_pa = alloc_page();
12         // 2. 找到父进程该页的物理地址
13         void* parent_page_pa =
14 (void*)lookup_pa(parent->pagetable, current_va);
15
16         // 3. 复制数据
17         memcpy(new_page_pa, parent_page_pa, PGSIZE);
18
19         // 4. 建立映射
20         user_vm_map(child->pagetable, current_va,
21 PGSIZE, (uint64)new_page_pa,
22             prot_to_type(PROT_WRITE |
23 PROT_READ, 1));
24     }
25
26     // 更新子进程的 mapped_info
27     int idx = child->total_mapped_region;
28     child->mapped_info[idx].va = start_va;
29     child->mapped_info[idx].npages = page_cnt;
30     child->mapped_info[idx].seg_type = DATA_SEGMENT;
31     child->total_mapped_region++;
32     break;
33 }
```

3.2.2 实现 wait 机制与阻塞队列

为了支持 wait 功能，在 kernel/sched.c 中引入了 wait_queue_head 并实现了以下辅助函数：

(1) insert_to_wait_queue: 将当前进程状态设为 BLOCKED，并将其插入等待队列链表。

```
01 void insert_to_wait_queue(process *proc)
02 {
03
04     proc->status = BLOCKED;
05     proc->queue_next = NULL;
06
07     if (wait_queue_head == NULL)
08     {
09         wait_queue_head = proc;
10         return;
11     }
12
13     // wait queue is not empty
14     process *ptr = wait_queue_head;
15     // 遍历到队尾
16     while (ptr->queue_next != NULL) {
17         if (ptr == proc) return; // already in queue
18         ptr = ptr->queue_next;
19     }
20
21     if (ptr == proc) return;
22     ptr->queue_next = proc;
23     return;
24 }
```

(2) wake_up: 当进程退出时调用。遍历等待队列，查找是否有父进程正在等待当前退出进程。若找到，将其状态改为 READY 并移入就绪队列。

```
01 void wake_up(process *curr_proc)
02 {
03     if (wait_queue_head == NULL)
04     {
05         return;
06     }
07
08     process *parent = curr_proc->parent;
09
10     // Case 1: 队头就是要唤醒的父进程
11     if (wait_queue_head == parent)
```

```

12  {
13      process *target = wait_queue_head;
14      wait_queue_head = wait_queue_head->queue_next;
15
16      target->status = READY;
17      insert_to_ready_queue(target);
18      return;
19  }
20
21  // Case 2: 遍历队列查找父进程
22  process *prev = wait_queue_head;
23  while (prev->queue_next != NULL)
24  {
25      if (prev->queue_next == parent)
26      {
27          process *target = prev->queue_next;
28          // 从等待队列移除
29          prev->queue_next = target->queue_next;
30
31          // 加入就绪队列
32          target->status = READY;
33          insert_to_ready_queue(target);
34          return;
35      }
36      prev = prev->queue_next;
37  }
38  }

```

(3) 在 kernel/syscall.c 中实现了 sys_user_wait:
调用 do_wait 检查是否有子进程已变为 ZOMBIE。

```

01  int do_wait(int pid) {
02      int target_pid = -1; // 默认返回值
03
04      // 遍历所有进程寻找子进程
05      for (int i = 0; i < NPROC; i++) {
06          // 如果不是当前进程的子进程，或者是未使用的槽位，直接
07          跳过
08          if (procs[i].parent != current || procs[i].status ==
09          FREE) {
10              continue;
11          }
12
13          // 如果指定了特定 PID 且不匹配，跳过

```

```

14         if (pid != -1 && procs[i].pid != pid) {
15             continue;
16         }
17
18         // 找到了符合条件的子进程
19         if (procs[i].status == ZOMBIE) {
20             target_pid = procs[i].pid;
21             procs[i].status = FREE;
22             procs[i].parent = NULL;
23             return target_pid;        // 成功回收, 返回 PID
24         } else {
25             // 找到了子进程但它还没死, 标记一下至少找到了一个
26             if (target_pid == -1) target_pid = 0;
27         }
28     }
29
30     return target_pid;
31 }

```

若子进程仍在运行, 则调用 `insert_to_wait_queue(current)` 将父进程阻塞, 并调用 `schedule()` 让出 CPU。

```

01 // 这里的逻辑配合 process.c 中的 do_wait 实现
02 ssize_t sys_user_wait(uint64 pid) {
03     int result;
04
05     // 循环等待, 直到子进程退出
06     while (1) {
07         result = do_wait(pid);
08
09         // 如果返回值不为 0, 说明有结果了
10         if (result != 0) {
11             return result;
12         }
13
14         // result 为 0, 说明子进程还在运行, 父进程需要阻塞等待
15         insert_to_wait_queue(current);
16         schedule();
17     }
18 }

```

3.2.3 实验结果

编译并运行 `spike ./obj/riscv-pke ./obj/app_wait`。如图 3.1 所示, 输出结果

显示孙子进程先退出（flag=2），唤醒子进程；子进程退出（flag=1），唤醒父进程；最后父进程退出（flag=0）。且各进程打印的 flag 值互不影响，证明数据段复制成功。

```
lq67677@angel:/mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ spike ./obj/riscv-pke ./obj/app_wait
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x000000008000b000, PKE kernel size: 0x000000000000b000 .
free physical memory address: [0x000000008000b000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _text is: 0x0000000080005000
kernel page table is on
Switch to user mode...
in alloc_proc. user frame 0x0000000087fb000, user stack 0x000000007ffff00, user kstack 0x0000000087fbb000
User application is loading.
Application: ./obj/app_wait
CODE_SEGMENT added at mapped info offset:4
DATA_SEGMENT added at mapped info offset:5
Application program entry point (virtual address): 0x0000000000010000
going to insert process 0 to ready queue.
going to schedule process 0 to run.
User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x0000000087fae000, user stack 0x000000007ffff00, user kstack 0x0000000087fad000
going to insert process 1 to ready queue.
going to schedule process 1 to run.
User call fork.
will fork a child from parent 1.
in alloc_proc. user frame 0x0000000087fa1000, user stack 0x000000007ffff00, user kstack 0x0000000087fa0000
going to insert process 2 to ready queue.
going to schedule process 2 to run.
Grandchild process end, flag = 2.
User exit with code:0.
going to insert process 1 to ready queue.
going to schedule process 1 to run.
Child process end, flag = 1.
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
Parent process end, flag = 0.
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

图 3.1 lab3_challenge1_wait 结果图

3.2.4 实验 git log 记录

下图展示了我在 lab3_challenge1_wait 实验的 git 日志。

```
lq67677@angel:/mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ git log
commit e39c326cf5f5886b87e5aeb01e4b854279bb5125 (HEAD -> lab3_challenge1_wait, origin/lab3_challenge1_wait)
Author: liuqi <3474856204@qq.com>
Date:   Wed Dec 24 23:19:07 2025 +0800

    修改优化lab3_challenge1

commit 59cab31169c5dd6da13159528b34c8ec540d99ce
Author: liuqi <3474856204@qq.com>
Date:   Fri Dec 19 16:08:11 2025 +0800

    解决头歌终端报错，头文件引用和上传不完全

commit b8055ef1e14c68b706622269d5a3efcfbf4271b9
Author: liuqi <3474856204@qq.com>
Date:   Fri Dec 19 15:42:14 2025 +0800

    lab3_challenge1还存在头歌提交行数限制问题。

commit c1a3dfef8a7ea895095f96e9f3bcbdd2cab2f2526
Merge: 18ef8d7 550b0dd
Author: liuqi <3474856204@qq.com>
Date:   Thu Dec 18 17:01:04 2025 +0800

    continue to work on lab3_challenge1
```

图 3.2 lab3_challenge1_wait 提交日志图

3.3 实验调试及心得

3.3.1 实验调试记录

1.父进程无法被唤醒问题

在初步实现时，我发现程序执行完子进程输出后就卡住了。原因是在 `sys_user_exit` 中，我仅调用了 `free_process` 释放资源，忘记调用 `wake_up`。导致子进程虽然变成了 ZOMBIE，但处于 BLOCKED 状态的父进程仍然在等待队列中，调度器无法将其选为 READY。

```
going to insert process 0 to ready queue.
going to schedule process 0 to run.
User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x000000087fae000, user stack 0x00000007ffff000, user kstack 0x000000087fad000
going to insert process 1 to ready queue.
going to schedule process 1 to run.
User call fork.
will fork a child from parent 1.
in alloc_proc. user frame 0x000000087fa1000, user stack 0x00000007ffff000, user kstack 0x000000087fa0000
going to insert process 2 to ready queue.
going to schedule process 2 to run.
Grandchild process end, flag = 2.
User exit with code:0.
ready queue empty, but process 0 is not in free/zombie state:3
ready queue empty, but process 1 is not in free/zombie state:3
Not handled: we should let system wait for unfinished processes.
```

图 3.3 父进程无法被唤醒问题示意图

2.数据段未隔离问题

在完善 `do_fork` 时，我最初为了图省事，对 `DATA_SEGMENT` 也使用了和 `CODE_SEGMENT` 一样的 `map_pages` 直接映射物理地址。

现象：运行 `app_wait` 时，父进程打印的 `flag` 变成了 1 或 2（被子进程修改了），而不是预期的 0。

修正：必须使用 `alloc_page + memcpy` 进行物理内存的深拷贝。

3.3.2 AI 辅助分析

1.关于 `do_fork` 中内存复制的必要性

在 `do_fork` 中，为什么 `CODE_SEGMENT` 可以直接 `map`，而 `DATA_SEGMENT` 必须 `alloc_page` 然后 `memcpy`？如果数据段也直接 `map` 会发生什么？

Show thinking ▼

这是一个非常经典且核心的操作系统内存管理问题，触及了**进程隔离 (Process Isolation)** 的本质。

1. 为什么 CODE_SEGMENT 可以直接 Map?

在 `kernel/process.c` 的 `do_fork` 实现中, 代码段的处理通常如下:

```
C
// kernel/process.c
user_vm_map(child->pagetable, code_va, PGSIZE, code_pa,
            prot_to_type(PROT_READ | PROT_EXEC, 1));
```

- **只读性 (Read-Only)**: 代码段存放的是机器指令, 在程序运行过程中, 指令本身是**不会被修改的** (除非涉及自修改代码, 但在现代 OS 保护模式下通常被禁止)。
- **共享安全性**: 既然内容不变, 那么父进程和子进程共享同一份物理内存是完全安全的。
- **节省资源**: 直接映射可以避免无谓的内存复制, 节省物理内存, 并加快 `fork` 的速度。

2. 为什么 DATA_SEGMENT 必须 alloc_page + memcpy ?

数据段 (包含全局变量、静态变量等) 在程序运行时是**可读可写 (Read-Write)** 的。

- **独立性要求**: 根据 `fork` 的语义, 子进程被创建后, 它应该拥有自己**独立的地址空间**。虽然子进程初始状态下的数据内容和父进程一模一样, 但它们在逻辑上是两个独立的副本。
- **物理复制**: 为了保证子进程修改变量时不影响父进程, 必须分配新的物理页 (`alloc_page`), 并将父进程的数据完整拷贝过来 (`memcpy`)。

3. 如果数据段也直接 Map 会发生什么?

如果你在 `do_fork` 中对 `DATA_SEGMENT` 也使用了直接映射 (即让父进程的虚拟地址指向同一个物理地址), 会发生严重的进程干扰。

图 3.4 AI 回答问题内存复制问题示意图

验证与分析: 我通过调试记录中的问题 2 验证了这一点。当我不进行 `memcpy` 时, 父子进程的变量确实相互干扰。这让我理解了 `fork` 的本质: 逻辑上是副本, 物理上可以通过不同策略实现, 但必须保证是独立的。

3.3.3 实验心得

本次挑战实验让我对进程生命周期有了代码级的认知。以前只知道 `wait` 会等待, 现在明白其底层是 `Running -> Blocked -> Ready` 的状态流转。特别是在处理 `sys_user_exit` 时, 我深刻体会到了系统调用之间的关系: `exit` 不仅仅是结束自己, 由于 `wait` 的存在, 它还肩负着“通知父进程”的责任。这种进程间的协作关系是操作系统设计的精髓。此外, 通过手动实现内存拷贝, 我复习了虚拟地址到物理地址的转换 (`lookup_pa`), 进一步巩固了 Lab2 的知识。

实验四 lab4_challenge1_relativepath

4.1 实验目的

深入理解虚拟文件系统（VFS）中路径解析的核心机制，掌握从路径字符串到目录项（dentry）的查找与遍历过程。掌握进程控制块（PCB）中文件管理结构的设计，特别是进程当前工作目录（CWD）的维护方法。通过实现 `pwd` 和 `cd` 系统调用，以及对 `.`（当前目录）和 `..`（父目录）的支持，增强 PKE 操作系统对用户态相对路径访问的支持。

4.2 实验内容

按照实验要求，修改 PKE 内核文件系统逻辑：使得 `open`, `opendir` 等涉及路径的系统调用能够识别以 `.` 或 `..` 开头的相对路径，以及直接以文件名开头的相对路径。

实现 `pwd` 功能：增加系统调用，允许用户获取当前进程工作目录的绝对路径。

实现 `cd` 功能：增加系统调用，允许用户修改当前进程的工作目录，并在内核中更新 PCB 的 `cwd` 指针。

4.2.1 核心路径解析逻辑改进

修改 `lookup_final_dentry` 函数，使其在解析路径时，首先判断路径类型。如果是绝对路径（以 `/` 开头），则从根目录 `vfs_root_dentry` 开始；如果是相对路径，则从 `current->pfiles->cwd` 开始。同时，在循环解析 `token` 时，增加对特殊目录项 `.` 和 `..` 的处理逻辑。

```
01 struct dentry *lookup_final_dentry(const char *path, struct
02 dentry **parent, char *miss_name) {
03     char path_copy[MAX_PATH_LEN];
04     strcpy(path_copy, path);
05
06     struct dentry *this;
07     if (path[0] == '/') {
08         this = vfs_root_dentry;
09     } else {
10         this = current->pfiles->cwd;
11     }
```

```

12
13 char *token = strtok(path_copy, "/");
14 if (token == NULL) {
15     *parent = this->parent;
16     return this;
17 }
18
19 while (token != NULL) {
20     *parent = this; // 当前节点成为下一轮的父节点
21
22     if (strcmp(token, ".") == 0) {
23         // 不改变 this
24     } else if (strcmp(token, "..") == 0) {
25         if (this->parent != NULL) this = this->parent;
26         else this = vfs_root_dentry;
27     } else {
28         // 查找子节点
29         struct dentry *found = hash_get_dentry((*parent),
30 token);
31         if (found == NULL) {
32             found = alloc_vfs_dentry(token, NULL, *parent);
33             struct vnode *found_vinode =
34 viop_lookup((*parent)->dentry_inode, found);
35
36             if (found_vinode == NULL) {
37                 free_page(found);
38                 strcpy(miss_name, token);
39                 return NULL;
40             }
41
42             // 处理 vnode 哈希
43             struct vnode *same_inode =
44 hash_get_vinode(found_vinode->sb, found_vinode->inum);
45             if (same_inode != NULL) {
46                 found->dentry_inode = same_inode;
47                 same_inode->ref++;
48                 free_page(found_vinode);
49             } else {
50                 found->dentry_inode = found_vinode;
51                 found_vinode->ref++;
52                 hash_put_vinode(found_vinode);
53             }
54             hash_put_dentry(found); // 此处若 found 地址不对齐
55 会报错

```

```

56     }
57     this = found; // 必须更新 this, 确保下一轮 token 在
found 下查找
    }
    token = strtok(NULL, "/");
    }
    return this;
}

```

4.2.2 实现目录切换系统调用

实现 `do_change_cwd` 函数。该函数首先调用 `lookup_final_dentry` 找到目标路径的 `dentry`，确认其存在且为目录类型后，更新当前进程的 `cwd` 指针。

```

01 int do_change_cwd(char *path) {
02     struct dentry *parent = (path[0] == '/') ?
03     vfs_root_dentry : current->pfiles->cwd;
04
05     char miss_name[MAX_PATH_LEN];
06     struct dentry *new_cwd = lookup_final_dentry(path,
07     &parent, miss_name);
08
09     if (new_cwd && new_cwd->dentry_inode->type == DIR_I) {
10         current->pfiles->cwd = new_cwd;
11         return 0;
12     }
    return -1;
}

```

4.2.3 系统调用注册

在 `do_syscall` 中增加 `SYS_user_rcwd` (read cwd) 和 `SYS_user_ccwd` (change cwd) 的分发逻辑。（代码实现简单，此处不展示。）

4.2.4 实验结果

编译并运行 `spike ./obj/riscv-pke ./obj/app_relativepath`。

如图 4.1 所示，输出结果分为三个阶段验证了相对路径功能的正确性：

目录切换 (Test 1)：结果显示进程成功调用 `change_cwd` 将当前工作目录从根目录 `/` 切换到了 `/RAMDISK0`，且 `read_cwd` (`pwd`) 返回的路径去除了宿主主机前缀，显示正确。

相对路径文件操作 (Test 2)：在 `/RAMDISK0` 目录下，进程能够识别 `./ramfile`

这种相对路径格式，并成功完成了文件的写入与读取，读写内容完全一致。

父目录跳转（Test 3）：进程执行 `cd ..` 后，`cwd` 正确变回了根目录 `/`，并能通过相对路径 `./hostfile.txt` 成功访问位于根目录下的主机文件。

最终输出 `All tests passed!`，证明内核中 `lookup_final_dentry` 对特殊目录项（`.` 和 `..`）的解析逻辑正确，且进程控制块（PCB）能正确维护当前工作目录的状态。

```
Switch to user mode...
in alloc_proc. user frame 0x0000000087f29000, user stack 0x000000007ffff000, user kstack 0x0000000087f28000
FS: created a file management struct for a process.
in alloc_proc. build proc_file_management successfully.
User application is loading.
Application: ./obj/app_relativepath
CODE_SEGMENT added at mapped info offset:4
DATA_SEGMENT added at mapped info offset:5
Application program entry point (virtual address): 0x00000000000100ec
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== Test 1: change current directory =====
cwd:/
change current directory to ./RAMDISK0
cwd:/RAMDISK0

===== Test 2: write/read file by relative path =====
write: ./ramfile
file descriptor fd: 0
write content:
hello world
read: ./ramfile
read content:
hello world

===== Test 3: Go to parent directory =====
cwd:/RAMDISK0
change current directory to ..
cwd:/
read: ./hostfile.txt
file descriptor fd: 0
read content:
This is an apple.
Apples are good for our health.

All tests passed!

User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

图 4.1 lab4_challenge1_relativepath 结果图

4.2.5 实验 git log 记录

下图展示了我在 lab4_challenge1_relativepath 实验的 git 日志。

```
lq67677@angel:/mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ git log
commit 7e19a4528c9e8d54bfff1bc631f732f5b5f17e59 (HEAD -> lab4_challenge1_relativepath, origin/lab4_challenge1_relativepath)
Author: liuqi <3474856204@qq.com>
Date: Wed Dec 24 22:22:57 2025 +0800

    my work on lab4_challenge1 is done.

commit 626200d93b672dad82edf048dfd44ada687f958a
Merge: b047d4e 99a368a
Author: liuqi <3474856204@qq.com>
Date: Wed Dec 24 21:06:36 2025 +0800

    continue to work on lab4_challenge1
```

图 4.2 lab4_challenge1_relativepath 提交日志图

4.3 实验调试及心得

本章节记录了在实验过程中遇到的两个典型问题。为了验证对代码逻辑的理解，我通过修改代码重现了这些错误场景，并记录了相应的报错信息。

4.3.1 实验调试记录

1. 相对路径查找失败问题

程序运行正常，但在执行 `cd ..` 返回上一级目录后，尝试打开当前目录下的文件（如 `./hostfile.txt`）失败，系统抛出 `cannot open a directory` 错误。如下图所示，这是因为查找之后忘记更新 `this` 指针了。

```
lq67677@angel:/mnt/c/Users/liuqi/Desktop/os_labs/lq-os-lab$ spike ./obj/riscv-pke ./obj/app_relativepath
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080010000, PKE kernel size: 0x000000000010000 .
free physical memory address: [0x0000000080010000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080008000
kernel page table is on
RAMDISK0: base address of RAMDISK0 is: 0x0000000087f35000
RFS: format RAMDISK0 done!
Switch to user mode...
in alloc_proc, user frame 0x0000000087f29000, user stack 0x000000007ffff000, user kstack 0x0000000087f28000
FS: created a file management struct for a process.
in alloc_proc, build proc_file_management successfully.
User application is loading.
Application: ./obj/app_relativepath
CODE_SEGMENT added at mapped info offset:4
DATA_SEGMENT added at mapped info offset:5
Application program entry point (virtual address): 0x00000000000100ec
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== Test 1: change current directory =====
cwd:/
change current directory to ./RAMDISK0
cwd:/

===== Test 2: write/read file by relative path =====
write: ./ramfile
vfs_open: cannot open a directory!
file descriptor fd: -1
do_write: cannot write file!

System is shutting down with exit code -1.
```

图 4.3 相对路径查找失败问题示意图

2. 输出格式错误问题

实现的 `pwd` 功能输出了包含宿主机物理路径前缀（如 `hostfs_root`）的路径，不符合虚拟文件系统的路径规范。加上相关的判断逻辑即可。


```

===== Test 1: change current directory =====
cwd:./hostfs_root
change current directory to ./RAMDISK0
cwd:./hostfs_root/RAMDISK0

===== Test 2: write/read file by relative path =====
write: ./ramfile
file descriptor fd: 0
write content:
hello world
read: ./ramfile
read content:
hello world

===== Test 3: Go to parent directory =====
cwd:./hostfs_root/RAMDISK0
change current directory to ..
cwd:./hostfs_root
read: ./hostfile.txt
file descriptor fd: 0
read content:
This is an apple.
Apples are good for our health.

All tests passed!

User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.

```

图 4.4 输出格式错误问题示意图

4.3.2 AI 辅助分析

1.关于相对路径查找失败的分析

我的代码在 Test 3 出错了。程序执行了 `cd ..` 回到根目录，然后尝试打开 `./hostfile.txt`，结果报错 `vfs_open: cannot open a directory!`。这是为什么？

◆ Show thinking ▼

这个报错非常有误导性，但它揭示了路径查找的一个核心逻辑漏洞。`vfs_open` 报错是因为它拿到的 `dentry` 指向了根目录 `/`，而不是你想要的文件。这通常意味着你的查找函数 `lookup_final_dentry` 在解析文件名时“原地踏步”了。

问题分析：请检查你的 `while(token != NULL)` 循环。当解析到最后一个 token（即文件名 `hostfile.txt`）时，你调用了查找函数并找到了正确的 `dentry`（我们记为 `found`），但是你是不是忘记将 `this` 指针更新为 `found`？如果忘记更新，循环结束时 `this` 依然指向上一级目录（也就是根目录），导致 `vfs_open` 以为你要打开根目录。

修正建议：在循环的 `else` 分支（处理普通文件名的部分）的最后，务必添加：

C 📄

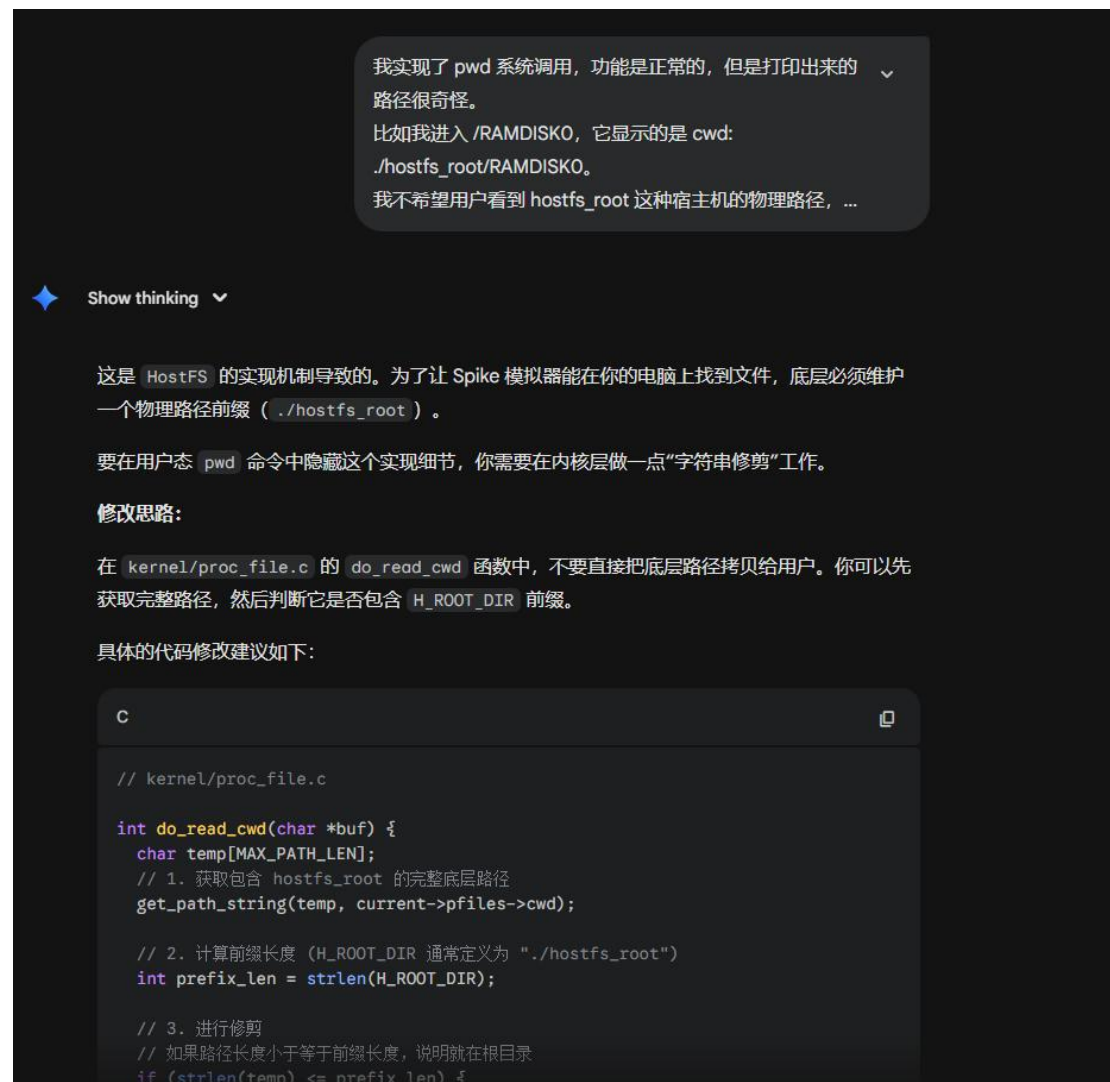
```
this = found; // 推进当前节点指针
```

图 4.5 AI 回答相对路径查找失败问题示意图

(1) 验证结果：对于上述的问题和 ai 的解答，验证过程已经在上节给出，在 `lookup_final_dentry` 的 `else` 分支末尾添加了 `this = found;` 代码。再次运行 `spike ./obj/riscv-pke ./obj/app_relativepath`, Test 3 成功读取了文件内容，输出了 All tests passed!。

(2) 观察与分析：`vfs_open` 接收到返回的 `dentry` 后，检查其 `inode` 类型，发现是 `DIR_I` (目录)，因此拒绝打开并报错。修复后，函数正确返回了文件 B 的 `dentry`，操作得以继续。

2.关于显示路径前缀的分析



我实现了 `pwd` 系统调用，功能是正常的，但是打印出来的路径很奇怪。
比如我进入 `/RAMDISK0`，它显示的是 `cwd: ./hostfs_root/RAMDISK0`。
我不希望用户看到 `hostfs_root` 这种宿主机的物理路径，...

Show thinking

这是 `HostFS` 的实现机制导致的。为了让 `Spike` 模拟器能在你的电脑上找到文件，底层必须维护一个物理路径前缀 (`./hostfs_root`)。

要在用户态 `pwd` 命令中隐藏这个实现细节，你需要在内核层做一点“字符串修剪”工作。

修改思路：

在 `kernel/proc_file.c` 的 `do_read_cwd` 函数中，不要直接把底层路径拷贝给用户。你可以先获取完整路径，然后判断它是否包含 `H_ROOT_DIR` 前缀。

具体的代码修改建议如下：

```
C
// kernel/proc_file.c

int do_read_cwd(char *buf) {
    char temp[MAX_PATH_LEN];
    // 1. 获取包含 hostfs_root 的完整底层路径
    get_path_string(temp, current->pfiles->cwd);

    // 2. 计算前缀长度 (H_ROOT_DIR 通常定义为 "./hostfs_root")
    int prefix_len = strlen(H_ROOT_DIR);

    // 3. 进行修剪
    // 如果路径长度小于等于前缀长度，说明就在根目录
    if (strlen(temp) <= prefix_len) {
```

图 4.6 AI 回答显示多余路径前缀问题示意图

(1) 验证结果：对于上述的问题和 ai 的解答，验证过程已经在上节给出，修改 `do_read_cwd` 函数，加入指针偏移逻辑即可。重新运行测试，`pwd` 的输出从 `cwd: ./hostfs_root/RAMDISK0` 变成了干净的 `cwd: /RAMDISK0`，完全符合预期。

(2) 观察与分析：在物理层面上，PKE 运行在 `Spike` 模拟器上，`HostFS` 实

际上是宿主机文件系统的一个子目录（`./hostfs_root`），内核必须使用这个完整路径才能进行文件 I/O。

4.3.3 实验心得

通过本次挑战实验，我完成了对 PKE 操作系统相对路径的支持，并得出以下感悟：

路径解析的复杂性：我认识到文件系统的路径解析不仅仅是字符串匹配，更是一个在目录树（Dentry Tree）中根据 Token 逐级遍历的过程。特殊字符 `.` 和 `..` 改变了遍历的方向，需要特殊处理。

CWD 的重要性：当前工作目录是相对路径解析的基石。在内核态正确维护 `cwd` 指针，并在进程切换或目录跳转时保持其一致性是非常关键的。

调试技巧：通过这次实验，我学会了如何通过 `panic` 信息（如 `cannot open a directory`）反推路径查找逻辑中的指针更新错误，以及如何处理宿主机路径与虚拟机路径之间的映射关系。

附录 完整 git log 记录

